PUC-SP

Prof.Eduardo Savino Gomes - Ciências de Dados e Inteligência Artificial

CIÊNCIAS DE DADOS E INTELIGÊNCIA ARTIFICIAL

SEMANA 3

CONSUMINDO DADOS DE API

AGENDA

- API – Application Program Interface
- JSON - JavaScript Object Notation
- Acessando URL em Python
- Acessando o site https://rapidapi.com/
- Consumindo API em Python

API – Application Program Interface

API - Application Program Interface (Interface de Programação de Aplicativos) é um conjunto de definições e protocolos para construir e integrar software de aplicativos.

As APIs permitem que seu produto ou serviço se comunique com outros produtos e serviços sem precisar saber como eles são implementados

API – Application Program Interface

Service Oriented Architecture (SOA)

- Arquitetura Orientada a Serviços (SOA)
  - Tipo de sistema distribuído
  - Os serviços devem cooperar para implementar uma única funcionalidade.
  - Habilita a chamada de objetos e serviços remotos (funções)
  - Fornece ferramentas para descoberta dinâmica de serviços, enfatizando a interoperabilidade.
  - Os componentes publicam suas funcionalidades em um registro centralizado e exportam uma API no nível da rede para que outros serviços possam acessá-los programaticamente
- Separar a lógica de negócios da lógica de apresentação.
- Projete um aplicativo como um conjunto de serviços.
- Os aplicativos de próxima geração resolvem dinamicamente as necessidades de serviço.

API – Application Program Interface

Arquitetura REST
(**RE**presentational **S**tate **T**ransfer)
Transferência de Estado Representacional

Definido no ano 2000, na tese de doutorado de Roy Fielding e é a sigla para *Representational State Transfer*: é um ***design de arquitetura*** construído para servir aplicações em rede.

A arquitetura REST combina os seguintes conceitos:

- Cliente-Servidor
- State Less (Sem Estado)
- Cache
- Código Sob Demanda
- Sistema em Camadas
- Interface Uniforme

Exemplo
https://rapidapi.com/apidojo/api/imdb8

API – Application Program Interface

IMDb FREEMIUM Verified ✓

By Api Dojo | Updated 31 minutes ago | | News, Media | Featured in Best Movie APIs

Popularity	Latency	Service Level	Health Check
9.9 / 10	1.172ms	100%	N/A

Endpoints About Tutorials Discussions Pricing Subscribed

IMDb API helps to query for all information about films, actors, characters,etc... as on official we Show more...

V1 (Current)

Search endpoints

- GET auto-complete
- GET v2/search
- POST v2/search-advance
- actor
- title
- news

GET auto-complete

Test Endpoint

Get auto complete suggestion by term or phrase

Personal Account
Savino

RapidAPI App: default-application_4719988
REQUIRED

Request URL: rapidapi.com
REQUIRED

Code Snippets Example Responses Results

(Node.js) Axios Copy Code

```
const axios = require('axios');

const options = {
  method: 'GET',
  url: 'https://imdb8.p.rapidapi.com/auto-complete',
  params: {q: 'game of thr'},
  headers: {
    'X-RapidAPI-Key': '7395c74fcdmshc55eca2cd2c9a8cp1901a1jsn9d9
900b7993',
    'X-RapidAPI-Host': 'imdb8.p.rapidapi.com'
  }
};

try {
    const response = await axios.request(options);
```

API – Application Program Interface
Formato JSON


```
b'{"error":false,
   "statusCode":200,
   "message":"OK",
   "data":{
           "lastChecked":"2022-09- 28T19:05:49.906Z“,
           "covid19Stats":
            [{"city":null,"province":"Alberta",
              "country":"Canada",
              "lastUpdate":"2022-09-28T04:23:01+00:00",
              "keyId":"Alberta,Canada",
              "confirmed":605757,
              "deaths":4872,
              "recovered":null},
             {"city":null,"province":"British Columbia",
"country":"Canada","lastUpdate":"2022-09-28T04:23:01+00:00","keyId":"British
Columbia, Canada","confirmed":384266,"deaths":4253,"recovered":null}}}'
```

JSON - JavaScript Object Notation

Fonte: https://www.json.org/

JSON (JavaScript Object Notation) é um formato leve de intercâmbio de dados. É fácil para um pessoa ler e escrever. É fácil para uma máquina analisar e gerar. Ele é baseado em um subconjunto do JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999. JSON é um formato de texto completamente independente de linguagem, mas usa convenções familiares aos programadores da família C de linguagens, incluindo C, C++, C#, Java, JavaScript, Perl, Python e muitos outros. Essas propriedades tornam o JSON uma linguagem de intercâmbio de dados ideal.

JSON é construído em duas estruturas:

- Uma coleção de pares nome/valor. Em várias linguagens, isso é realizado como um objeto, registro, estrutura, dicionário, tabela de hash, lista de chaves ou matriz associativa.
- Uma lista ordenada de valores. Na maioria das linguagens, isso é realizado como uma matriz, vetor, lista ou sequência.

JSON - JavaScript Object Notation

Fonte: https://www.json.org/

```
json
    element

value
    object
    array
    string
    number
    "true"
    "false"
    "null"

object
    '{' ws '}'
    '{' members '}'

members
    member
    member ',' members

member
    ws string ws ':' element

array
    '[' ws ']'
    '[' elements ']'

elements
    element
    element ',' elements
```


```
[{
  "id": 1,
  "first_name": "Jeanette",
  "last_name": "Penddreth",
  "email": "jpenddreth0@census.gov",
  "gender": "Female",
  "ip_address": "26.58.193.2"
}, {
  "id": 2,
  "first_name": "Giavani",
  "last_name": "Frediani",
  "email": "gfrediani1@senate.gov",
  "gender": "Male",
  "ip_address": "229.179.4.212"
}, {
  "id": 3,
  "first_name": "Noell",
  "last_name": "Bea",
  "email": "nbea2@imageshack.us",
  "gender": "Female",
  "ip_address": "180.66.162.255"
}]
```

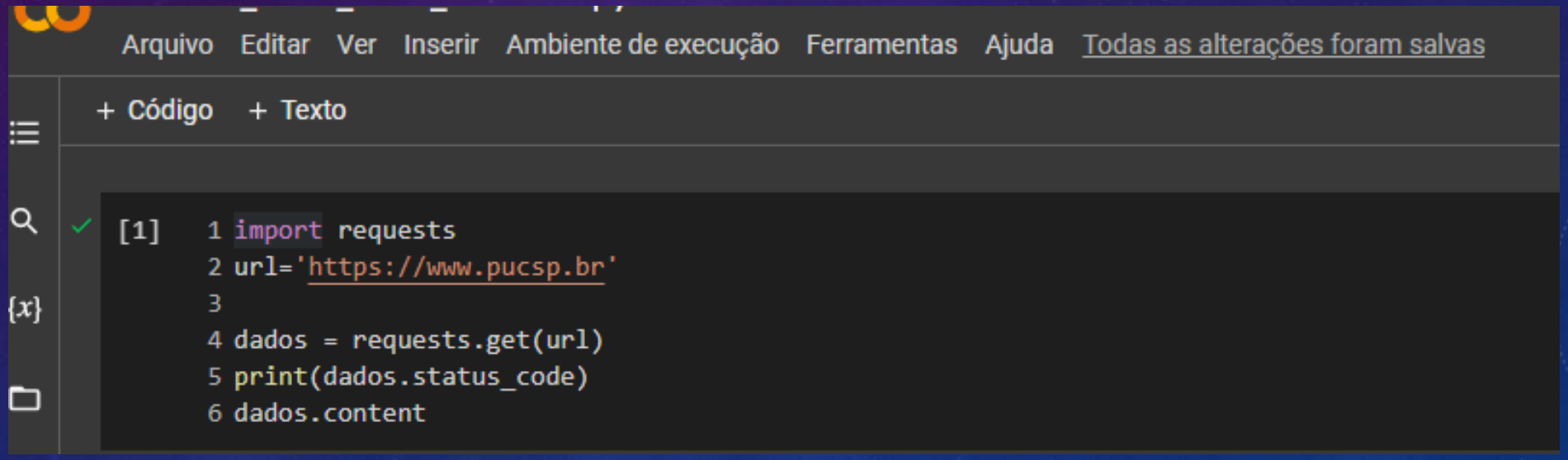
```
{
  "colors": [
    {
      "color": "black",
      "category": "hue",
      "type": "primary",
      "code": {
        "rgba": [255,255,255,1],
        "hex": "#000"
      }
    },
    {
      "color": "white",
      "category": "value",
      "code": {
        "rgba": [0,0,0,1],
        "hex": "#FFF"
      }
    },
]
}
```

```
{
  "beige": "#f5f5dc",
  "bisque": "#ffe4c4",
  "black": "#000000",
  "blanchedalmond": "#ffebcd",
  "blue": "#0000ff",
  "blueviolet": "#8a2be2",
  "brown": "#a52a2a",
}
```

```
{
  "beige": [245, 245, 220, 1],
  "bisque": [255, 228, 196, 1],
  "black": [0, 0, 0, 1],
  "blanchedalmond": [255, 235, 205, 1],
  "blue": [0, 0, 255, 1],
  "blueviolet": [138, 43, 226, 1],
  "brown": [165, 42, 42, 1],
}
```

Acessando URL em Python

Para fazer uma requisição HTTP ou HTTPS em Python é necessário utilizar a biblioteca ***requests***. A biblioteca *requests* tem diversas funções que facilitam comunicação via HTTP ou HTTPS, conforme exemplo abaixo:



Consumindo API em Python

No exemplo abaixo utiliza-se a o método requests.get() para consumir dados de uma API da rapidapi.com, uma API com dados do Mercado Financeiro oferecido pela Bloomberg

```
1 import requests
2 url='https://bloomberg-market-and-financial-news.p.rapidapi.com/market/get-movers'
3 dados = requests.get(url,params= {"id": 'nky:ind', "template": 'INDEX'}, headers= {
4     'X-RapidAPI-Key': '7395c74fcdmshc55eca2cd2c9a8cp1901a1jsn9d93900b7993',
5     'X-RapidAPI-Host': 'bloomberg-market-and-financial-news.p.rapidapi.com'
6   })
7 dados.content
```

Consumindo API em Python

No código abaixo identificamos os elementos nomes do JSON enviado. Execute o código abaixo e tente analisar o resultado

```python
import json
dados_dict=json.loads(dados.content)
dados_dict.keys()
```

```
dict_keys(['active', 'laggards', 'leaders'])
```

Consumindo API em Python

No código abaixo identificamos os dados ACTIVE

```python
import pandas as pd
df_active=pd.DataFrame(dados_dict["active"])
df_active
```

currency	resourceSubtype	region	ticker	...	netChange	lastPriceTime	pctChange1M	yearHigh	dayHigh	volume	yearLow	dayLow	pctChangeYTD	pctChange
JPY	Public	APAC	9432:jp	...	1	1709877600	3	193	186	194965400.0	155	183	8	0.76
JPY	Public	APAC	8306:jp	...	16	1709877600	18	1651	1651	132216100.0	802	1607	35	0.96
JPY	Public	APAC	9501:jp	...	14	1709877600	8	895	808	57081600.0	440	786	9	1.73
JPY	Public	APAC	7201:jp	...	-3	1709877600	-0	712	555	42270400.0	450	542	-0	-0.58
JPY	Public	APAC	7203:jp	...	-50	1709877600	9	3811	3665	39890100.0	1764	3595	39	-1.37
JPY	Public	APAC	8411:jp	...	152	1709877600	18	3155	3155	36524700.0	1744	3027	31	5.05

Consumindo API em Python

Queremos analisa o percentual de variação do papel (coluna pctChange) para verificar qual papel está mais volátil.
Nesse caso os dados vieram em formato TEXTO, então precisamos converter para FLOAT

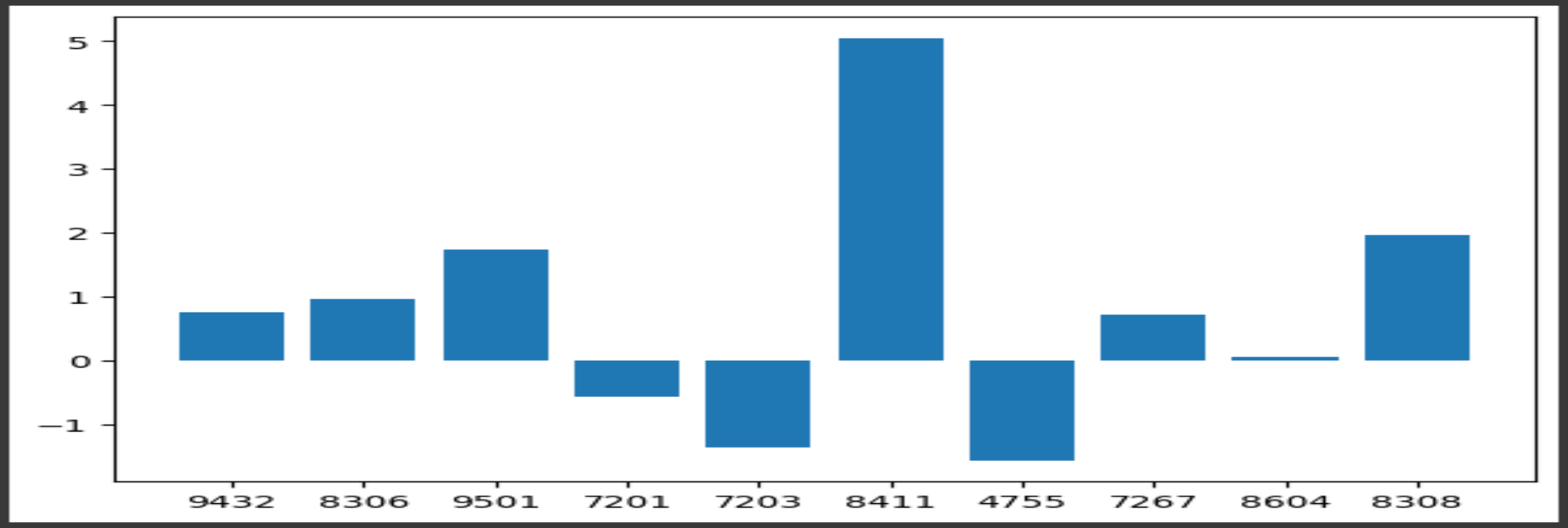
```python
from numpy.lib.function_base import append
float(df_active.pctChange[0])
pctChangeFloat=[]
for i in range(len(df_active.pctChange)):
  pctChangeFloat.append(float(df_active.pctChange[i]))

df_active["pct"]=pctChangeFloat
df_active["pct"]
```

```
0    0.76
1    0.96
2    1.73
3   -0.58
4   -1.37
5    5.05
6   -1.57
7    0.71
8    0.05
9    1.96
Name: pct, dtype: float64
```

Consumindo API em Python

Com os dados convertidos podemos apresentar um gráfico que ilustre a volatilidade do valor do papel

```python
import matplotlib.pyplot as plt
eixoX=df_active["symbol"]
eixoY=df_active["pct"]
plt.figure(figsize=(7,5))
plt.bar(eixoX,eixoY)
plt.show()
```



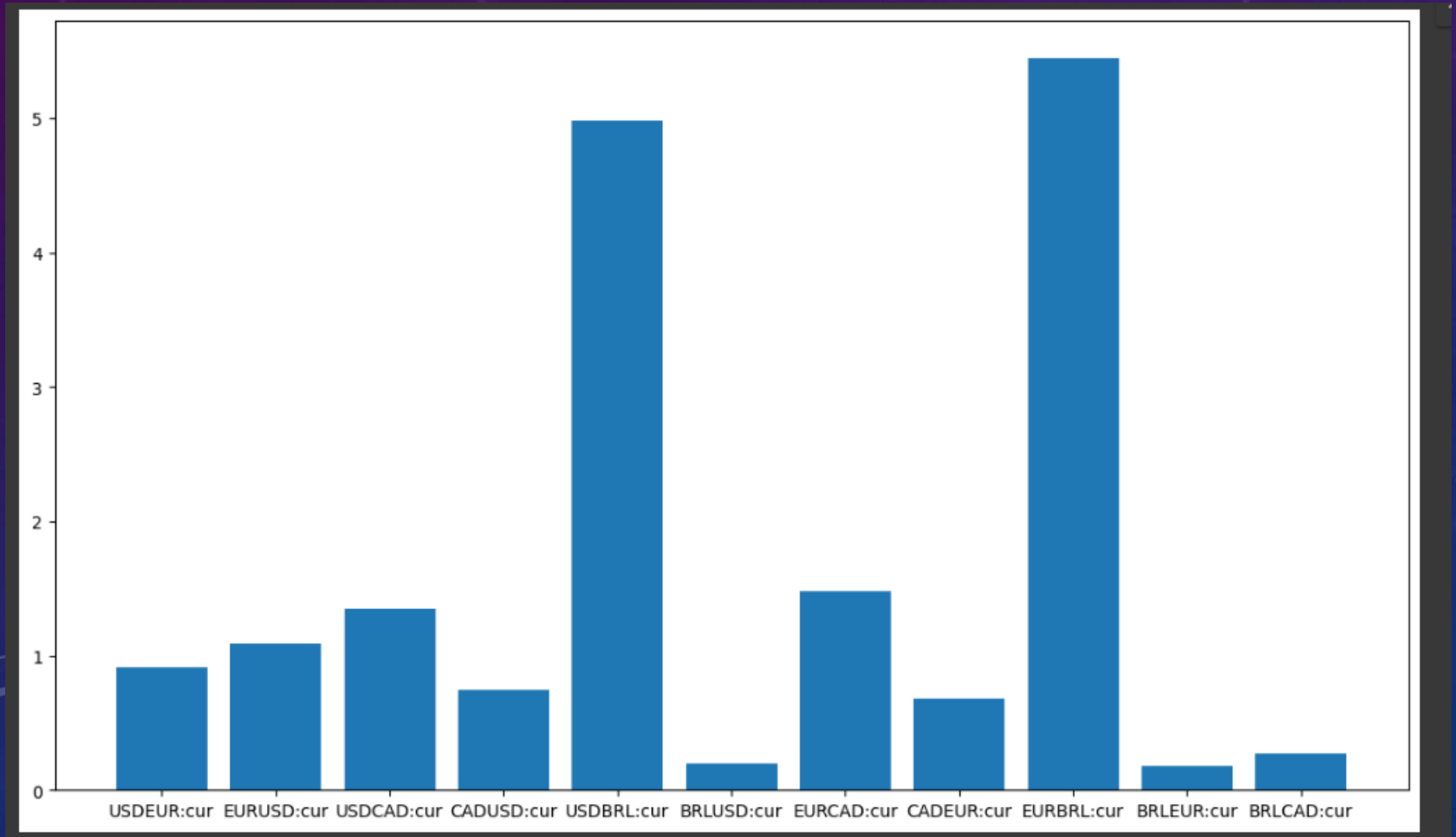
Consumindo API em Python

Considere agora o acesso a seguinte API

```
import requests
url='https://bb-finance.p.rapidapi.com/market/get-cross-currencies'
dados = requests.get(url,params= {"id": 'brl,cad,eur,usd'}, headers= {
    'X-RapidAPI-Key': '7395c74fcdmshc55eca2cd2c9a8cp1901a1jsn9d93900b7993',
     'X-RapidAPI-Host': 'bb-finance.p.rapidapi.com'
  })
dados.content
```

Consumindo API em Python

Faça um estudo dos dados apresentados e tente, com sua equipe apresentar o gráfico abaixo:



Esse gráfico apresenta o valor entre as moedas. A primeira coluna mostra a relação USDxEUR , já a segunda EURxUSD

Consumindo API em Python

A API **python-bcb** é uma interface em Python estruturada para obter informações da API de dados abertos do Banco Central do Brasil.

O projeto de Dados Abertos do Banco Central do Brasil disponibiliza diversas APIs provendo acesso direto a dados de:

- Moedas
- Taxas de Juros
- Índices de preços
- Informações de Instituições Financeiras
- Expectativas do Mercado (Expectativas do Boletim FOCUS)
- E muito outros …

O pacote python-bcb implementa interfaces para algumas APIs disponibilizadas pelo Banco Central de forma que o resultado das consultas, na maioria dos casos, é um DataFrame pandas formatado com os dados.

FONTE: https://wilsonfreitas.github.io/python-bcb/

Consumindo API em Python

APIs implementadas

sgs

Utiliza o webservice do SGS (Sistema Gerenciador de Séries Temporais) para obter os dados. Diversas séries estão disponíveis no SGS: taxas de juros, índices de preços, indicadores econômicos, …, e com um simples chamado da função bcb.sgs.get() é possível tê-las em um DataFrame pandas.

Conversor de Moedas

Implementado no módulo currency, um conjunto de funções que realiza webscraping no site do Conversor de Moedas do Banco Central, possível obter séries temporais de frequência diária de diversas moedas.

Moedas OData

O Banco Central disponibiliza diversas informações em APIs que seguem o padrão OData <https://odata.org>. A classe bcb.PTAX implementa uma API OData que entrega os boletins diários de taxas de câmbio do Banco Central. Esta API entrega mais informações do que o que é obtido no Conversor de Moedas.

FONTE: https://wilsonfreitas.github.io/python-bcb/

Consumindo API em Python

APIs implementadas

Expectativas

A API de Expectativas de Mercado traz todas as estatísticas das variáveis macroeconômicas fornecidos por um conjuto de instituições do mercado financeiro. A classe bcb.Expectativas implementa essa interface no padrão OData.

Taxas de Juros

API que retorna as taxas de juros de operações de crédito por instituição financeira (médias dos últimos 5 dias). A classe bcb.TaxaJuros implementa essa interface.

ODataAPI

O BCB disponibiliza diversas APIs que seguem a especificação OData. Algumas APIs mais utilizadas como as bcb.PTAX e bcb.Expectativas possuem uma classe específica, para as APIs menos utilizadas, é possível utilizar a classe bcb.ODataAPI para acessar a API. Toda API que segue a especificação OData possui uma URL de acesso, esta URL é passada para a classe bcb.ODataAPI e o objeto criado dá total acesso a API.

FONTE: https://wilsonfreitas.github.io/python-bcb/

Consumindo API em Python

Instalando a APY python-bcb

Abra um novo notebook no Colab e digite o código abaixo

```
1 !pip install python-bcb
2
```

O **pip** é um gerenciador de pacotes para Python que permite instalar, atualizar e remover bibliotecas para programação em Python.,
O **pip** instala também dependências adicionais que um pacote pode exigir.;

Consumindo API em Python

Utilizando python-bcb

Crie uma célula nova e execute o código abaixo

```python
from bcb import sgs
selic = sgs.get({'selic':432}, start='2025-01-01')
df=selic.reset_index()
print(df)
```

```
         Date  selic
0  2025-01-01  12.25
1  2025-01-02  12.25
2  2025-01-03  12.25
3  2025-01-04  12.25
4  2025-01-05  12.25
5  2025-01-06  12.25
6  2025-01-07  12.25
7  2025-01-08  12.25
8  2025-01-09  12.25
9  2025-01-10  12.25
10 2025-01-11  12.25
11 2025-01-12  12.25
12 2025-01-13  12.25
13 2025-01-14  12.25
14 2025-01-15  12.25
15 2025-01-16  12.25
16 2025-01-17  12.25
```

A biblioteca **sgs** utiliza o webservice do SGS (Sistema Gerenciador de Séries Temporais) disponibilizado pelo BACEN

O método **get()** retorna uma Dataframe (Pandas) com a série solicitada, no caso 432 (Selic)

O parâmetro `{'selic':432}` indica a série 423, com a coluna de nome arbitrário ‘selic’,

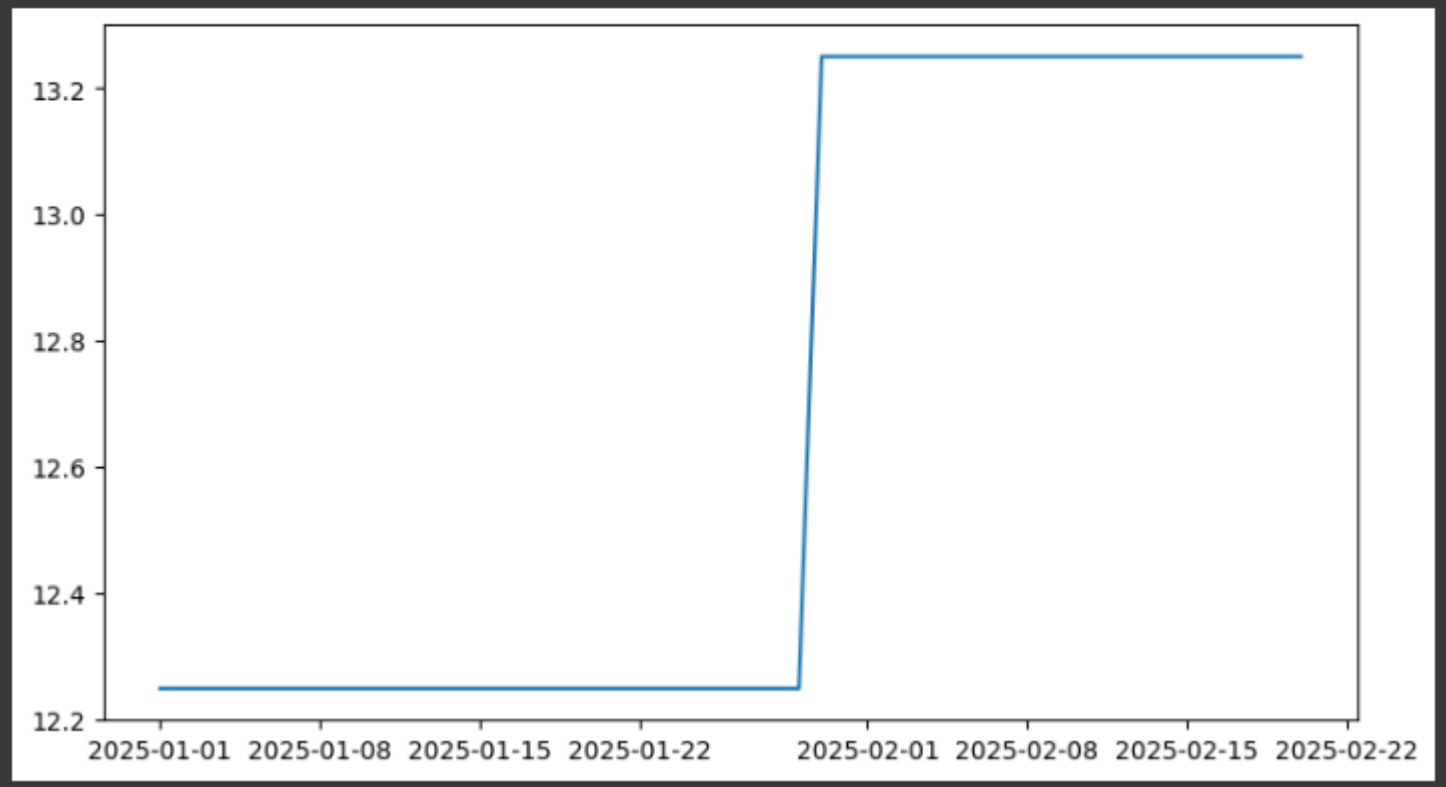
O parâmetro `start='2025-01-01'}` Determina a data inicial que queremos os dados da série

Consumindo API em Python

Utilizando python-bcb

Crie uma célula nova e execute o código abaixo

```
import matplotlib.pyplot as plt
plt.figure( figsize=(9, 5))
plt.plot(df.Date, df.selic)
plt.show()
```



Imprimimos um gráfico de linha utilizando a série da taxa selic

Altere a data start e veja como fica o gráfico

Consumindo API em Python

Utilizando python-bcb

É possível pegar ´várias séries de uma única vez confrne código abaixo

```python
serie_hist = sgs.get({'selic': 432,'igpm':189,'ipca':433,'inpc':188, },start='2024-01-01')
serie_hist = serie_hist.dropna()
serie_hist=serie_hist.reset_index()
print(serie_hist)
```

```
         Date  selic  igpm  ipca  inpc
0  2024-01-01  11.75  0.07  0.42  0.57
1  2024-02-01  11.25 -0.52  0.83  0.81
2  2024-03-01  11.25 -0.47  0.16  0.19
3  2024-04-01  10.75  0.31  0.38  0.37
4  2024-05-01  10.75  0.89  0.46  0.46
5  2024-06-01  10.50  0.81  0.21  0.25
6  2024-07-01  10.50  0.61  0.38  0.26
7  2024-08-01  10.50  0.29 -0.02 -0.14
8  2024-09-01  10.50  0.62  0.44  0.48
9  2024-10-01  10.75  1.52  0.56  0.61
10 2024-11-01  10.75  1.30  0.39  0.33
11 2024-12-01  11.25  0.94  0.52  0.48
12 2025-01-01  12.25  0.27  0.16  0.00
```

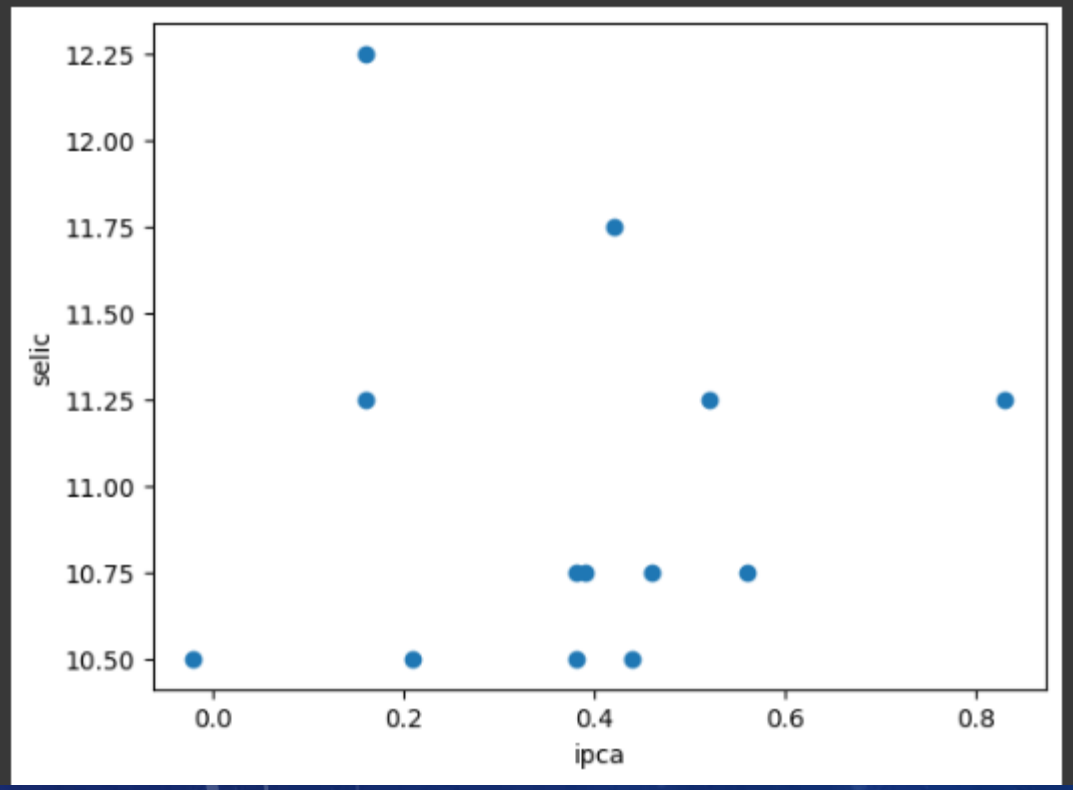
O método **dropna()** exclui linhas com valor nulo

Consumindo API em Python

Utilizando python-bcb

Imprimindo gráfico

```
plt.scatter(serie_hist.ipca, serie_hist.selic)
plt.ylabel('selic')
plt.xlabel('ipca')
plt.show()
```



Altere a data para comparar a relação entre outros índices de inflação

Consumindo API em Python

Utilizando python-bcb

Salvando um Dataframe em um arquivo .csv ou Excel

```
df.to_csv('selic.csv')
df.to_excel('selic.xlsx')
```

Consumindo API em Python

Utilizando python-bcb

O comando get() tem a seguinte sintaxe

sgs.get(codes, start=None, end=None, last=0, multi=True, freq=None)
Retorna um DataFrame pandas com séries temporais obtidas do SGS.
Parâmetros:
codes ({int, List[int], List[str], Dict[str:int]}) –
Este argumento pode ser uma das opções:
int : código da série temporal
list ou tuple : lista ou tupla com códigos
list ou tuple : lista ou tupla com pares ('nome', código)
dict : dicionário com pares {'nome': código}

Com códigos numéricos é interessante utilizar os nomes com os códigos para definir os nomes nas colunas das séries temporais.

Consumindo API em Python

Utilizando python-bcb

start (str, int, date, datetime, Timestamp) – Data de início da série. Interpreta diferentes tipos e formatos de datas.

end (string, int, date, datetime, Timestamp) – Data final da série. Interpreta diferentes tipos e formatos de datas.

last (int) – Retorna os últimos last elementos disponíveis da série temporal solicitada. Se last for maior que 0 (zero) os argumentos start e end são ignorados.

multi (bool) – Define se, quando mais de 1 série for solicitada, a função retorna uma série multivariada ou uma lista com séries univariadas.

freq (str) – Define a frequência a ser utilizada na série temporal

Retorna:
DataFrame – série temporal univariada ou multivariada, quando solicitado mais de uma série (parâmetro multi=True).

list – lista com séries temporais univariadas, quando solicitado mais de uma série (parâmetro multi=False).

Consumindo API em Python

Documentação completa no link https://wilsonfreitas.github.io/python-bcb/api.html

wilsonfreitas.github.io/python-bcb/api.html

documentação python-bcb

Buscar

SGS

Dados de Inadimplência de Operações de Crédito

Moedas

Expectativas

Taxas de Juros

OData

API

API

Módulo `bcb.sgs`

`bcb.sgs.get(codes, start=None, end=None, last=0, multi=True, freq=None)`

Retorna um DataFrame pandas com séries temporais obtidas do SGS.

PARÂMETROS:

- **codes** (*{int, List[int], List[str], Dict[str:int]}*) –

  Este argumento pode ser uma das opções:

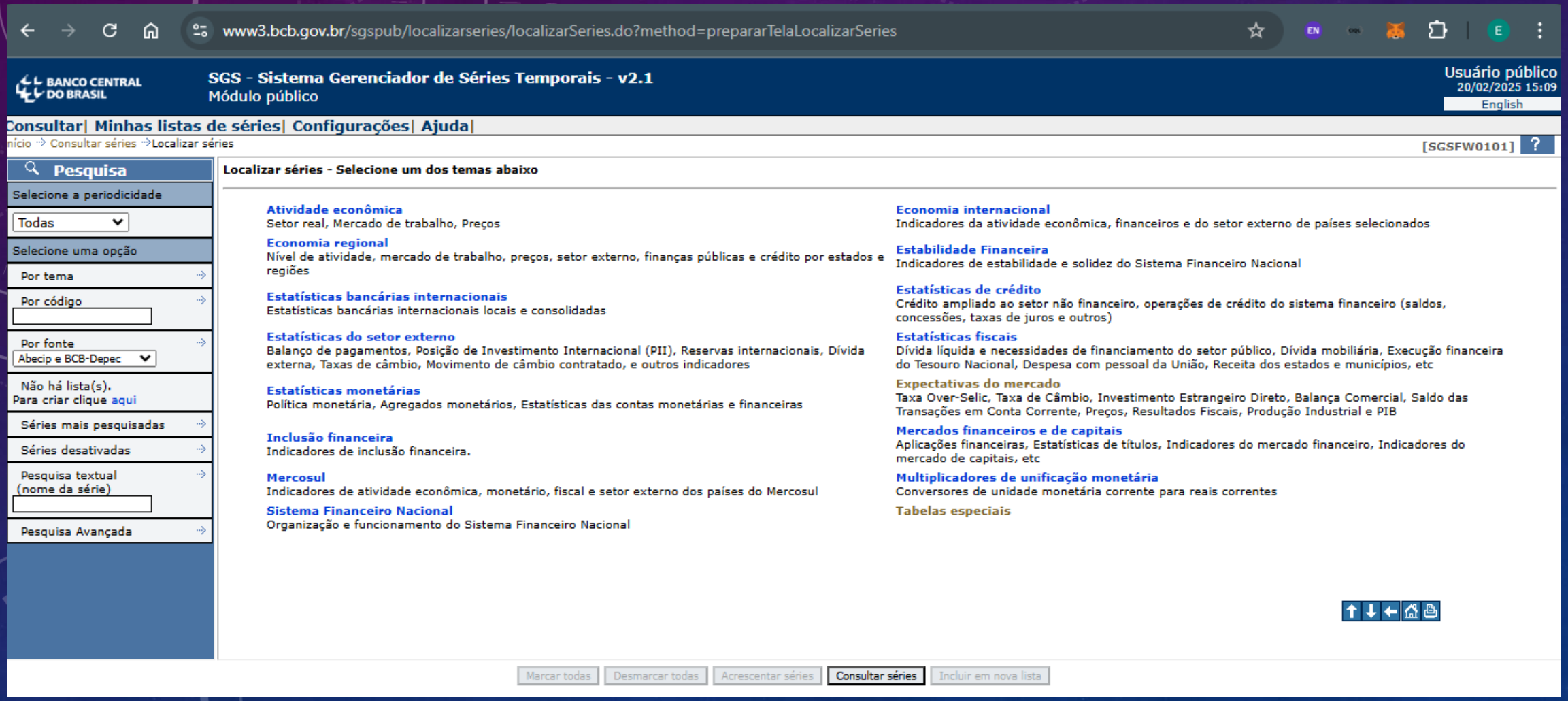
  - `int` : código da série temporal
  - `list` ou `tuple` : lista ou tupla com códigos
  - `list` ou `tuple` : lista ou tupla com pares `('nome', código)`
  - `dict` : dicionário com pares `{'nome': código}`

  Com códigos numéricos é interessante utilizar os nomes com os códigos para definir os nomes nas colunas das séries temporais.

- **start** (*str, int, date, datetime, Timestamp*) – Data de início da série. Interpreta diferentes tipos e formatos de datas.
- **end** (*string, int, date, datetime, Timestamp*) – Data final da série. Interpreta diferentes tipos e formatos de datas.
- **last** (*int*) – Retorna os últimos `last` elementos disponíveis da série temporal solicitada. Se `last`

Consumindo API em Python

Para localizar os códigos das séries do Banco Central acesse o link
https://www3.bcb.gov.br/sgspub/localizarseries/localizarSeries.do?method=prepararTelaLocalizarSeries



www.gov.br/conecta/catalogo/

conecta gov.br Ministério da Gestão e da Inovação em Serviços Públicos

vLibras Entrar

≡ Catálogo de APIs Governamentais

O que você procura?

Conecta - APIs de Governo para Governo

Por que solicitar ao cidadão informações que o governo já possui? Integre seu serviço público para obter as informações de maneira automática! É um direito do cidadão (Lei 13.726/2018) e uma economia para o governo!
O Catálogo de API Governamentais é uma iniciativa do Programa Conecta gov.br!

Conheça mais sobre o Conecta

https://www.gov.br/conecta/catalogo/

www.gov.br/conecta/catalogo/

conecta gov.br

Catálogo de APIs Governamentais

vLibras | Entrar

- APIS disponíveis na Plataforma do Conecta

Essas APIs estão disponíveis para acesso pela Plataforma do Conecta

- API Benefícios Previdenciários
- API CADIN CONSULTA / CONTRATANTE
- API CADIN CREDORA
- API de Qualificação de Endereços
- Benefício de Prestação Continuada - BPC
- Cadastro Base de Endereço (CEP)
- CadÚnico Serviços
- Cadúnico Serviços - Indicadores Familiares
- CadÚnico Serviços - Dados Familiares
- CAF - Cadastro Nacional da Agricultura Familiar – Consulta Organização Produtiva
- CAF - Cadastro Nacional da Agricultura Familiar – Serviços
- CAF - Cadastro Nacional da Agricultura Familiar – Unidade Familiar (UFPA)
- Carteira de documentos gov.br - Wallet
- Certidão de Antecedentes Criminais
- Compartilhamento de Dados de Adesão ao SINAPIR
- Consulta Certidão Negativa de Débitos (CND)